Grassroots Feeds and Followers

ABSTRACT

Grassroots platforms aim to offer an egalitarian and democratic alternative to incumbent global platforms, owned, operated and governed by their participants. Operationally, a grassroots platform consist of people and their smartphones, running the grassroots app. Conceptually, a platforms is grassroots if it can multiple volitionally-forming independent instances that may coalesce by mutual consent. Grassroots social networks that support direct messages, groups, child-safe social networking, and social networking with accountable AI agents and digital provenance have been presented previously.

Here we are concerned with grassroots feeds and followers, aiming to achieve the same functionality, say, of Twitter, but with a grassroots platforms. We present a grassroots social contract for feeds and followers, its implementation, analyse it via stochastic simulations, and discuss optimisation and fault-resilience.

CCS CONCEPTS

• **Computer systems organization** → **Peer-to-peer architectures**; • **Theory of computation** → **Distributed computing models**; • **Security and privacy** → *Social aspects of security and privacy*; • **Human-centered computing** → *Social networking sites.*

KEYWORDS

Grassroots Platforms, Grassroots Social Contracts, Volition-Guarded Transactions, Child Safety, Social Networks, GLP

1 INTRODUCTION

[TBD: The introduction is derived from the abstract once the abstract is agreed (style guide, "How a Paper Begins").]

Paper outline. Section 2 presents grassroots contracts and how one becomes an app, and the grassroots social graph and the directed overlay, which the contract extends. Section 3 presents the contract for feeds and followers, its speech-act schemas, its properties, GFF-I, an implementable restriction of it, and its app; Section 4 GFF-PS, its path-shortening restriction, with the paths on a random social graph; Section 5 GFF-FR, its fault-resilient restriction, and GFF-FR-BC, which extends it with back-channels. Section 6 discusses related work and Section 7 concludes. Appendix A recalls the mathematical foundations, Appendix C certifies the contract, and Appendix D holds the proofs.

2 GRASSROOTS CONTRACTS

A grassroots contract [6, 17] is a text in natural language, organised in clauses its participants undertake towards one another. Each clause describes an act — who takes it, who must agree to it, and what it changes — or binding the agents without describing one. Acquaintance is an act of two agents, both of whom must agree, as is befriending; ending a friendship is an act of one. The contract defines the platform: there is no server, no operator and no account, only the people who adopted the contract and their smartphones [45, 47]. Each contract is presented in a box, a clause is cited by the contract's name and its number, as GSG:1, and a contract that extends another cites its clauses and does not restate them. Each clause opens with its legal modality, as the box below sets out [17]. A contract also carries *pragmas*, which informs the user-interface design of the app: a line naming a tab of the app and what it lists, and, for each clause, a line naming the person who proposes its act, a button for each act they propose, in brackets, bearing its act's name, and any field they fill, in angle brackets, then an indented line for each other person asked, their name and the question put to them [17]; a line is named by a clause of the contract or of one it extends, as a feed's items are posted by GFF:1. The pragmas of the contracts this paper quotes are in the cited papers as well as in the repository https://github.com/EShapiro2/GLP; those of GFF end its box (Section 3).

Legal Modalities of Contract Clauses [17]

■ **Enforced:** no act of the contract can breach it.
◆ **Attested:** an agent may breach it, and the contract records the breach.
○ **Undertaken:** its performance lies outside the contract, which records only the undertaking.

Agents running code that correctly implements the contract cannot breach an enforced clause. Different parts of a clause may carry different modalities, in which case the clause carries the weakest modality among them.

A grassroots contract becomes a running app through the abstraction cascade [17, 24]: its clauses are rendered as speech-act schemas, which denote volition-guarded multiagent atomic transactions [30]; these are implemented in volition-guarded GLP [24], compiled to GLP [48], and run as a mini-app of the grassroots app [13] on each person's phone. All source code and all artefacts, including the contracts and the artefacts the abstraction cascade produces from them, are in the public repository https://github.com/EShapiro2/GLP.

2.1 Grassroots Keys and Knowledge

A *person* is a legally culpable entity: a natural person of age, also referred to as an *adult*, or a legal person of good standing, such as a business. A person takes the pronoun 'they'. An *agent* is a person or an entity that may act but is not legally culpable, such as a child or a computational (AI) agent, whose identity is a *ward* [49]. Alice, Bob, Carol and Dave name distinct agents, and each contract declares what kind of agent each is.

GKK: Grassroots Keys and Knowledge [17]

Alice, Bob and Carol are persons.

1. ○ *Identity.* Alice may establish their identity once by choosing a unique key pair. They make themselves known by the public key, sign their acts with the private key, and are accountable for every act signed with it.
2. ■ *Acquaintance.* Alice and Bob may come to know each other by an act of both.

3. ■ *Introduction.* If Alice and Bob know each other and Alice knows Carol, then Alice can make Carol known to Bob, with Bob's consent.

GKK is the base contract, which every grassroots contract extends [17]; a key pair chosen at random is unique with high probability, and its uniqueness is what the agent undertakes. An agent *knows* another when they hold the other's public key. Acquaintance, an act of both, makes two agents know each other; an introduction, an act of the introducer and the agent introduced to, makes that agent know the one introduced, who takes no part; knowing is never undone.

2.2 The Grassroots Social Graph

GSG: Grassroots Social Graph [17]

Alice and Bob are persons.

1. ◆ *Friendship.* Alice and Bob, who know each other personally and are not friends, may jointly decide to become friends.
2. ■ *Ending a friendship.* Alice may end their friendship with Bob.

GSG is the grassroots social graph [17, 46], which adds friendship to GKK; every grassroots platform that rests on friendship extends it, and coins extend GKK differently [44]. Two agents befriend only if they know each other. The social graph is stored distributively under the control of the people themselves, with each person storing the local neighbourhood pertaining to them, and no third party having access unless explicitly granted.

2.3 The Grassroots Directed Overlay

GDO: Grassroots Directed Overlay

GDO extends GSG [17].
Alice, Bob and Carol are persons.

1. ■ *Pointing to friend.* Alice, who does not point to their friend Bob, may point to Bob.
2. ■ *Transitive pointing.* Alice, who knows Carol and points to Bob who points to Carol, may point to Carol.
3. ■ *Ceasing to point.* Alice, who points to Bob, may cease pointing to Bob.

GDO is the grassroots directed overlay, of work in preparation, which adds pointing to GSG; the text in the box is the repository's copy, `programs/lbm/GDO.txt`, where its pragmas are. A person points to a friend (GDO:1), and to a person they know whom one they point to points to (GDO:2); the pointings form a directed graph, the *overlay*. A pointing is an act of the person who points and is recorded in their state; the person pointed to does not will it and keeps no record of it.

3 GRASSROOTS FEEDS AND FOLLOWERS

3.1 The Contract

The contract for feeds and followers is built on two contracts. GDO-L, the grassroots directed overlay with labels, extends GDO (Section 2.3) with labeled pointers: a pointer may be labeled by a person, a pointer labeled by Dave being a Dave-pointer, and Alice may have many pointers to Bob, labeled by different persons. GFF extends GDO-L: a feed is named by its author, and Alice follows Carol's feed through Bob when Alice has a Carol-pointer to Bob. The text in each box is the repository's copy, `programs/gff/GDO-L.txt` and `programs/gff/GFF.txt`.

GDO-L: Grassroots Directed Overlay with Labels

GDO-L extends GDO (GRH, Grassroots Ride-Hailing, in preparation).
Alice, Bob, Carol and Dave are persons, Carol and Dave possibly one person. A pointer may be labeled by a person, and a pointer labeled by Dave is a Dave-pointer; Alice may have many pointers to Bob, labeled by different persons. Friends Alice and Bob have a Bob-pointer from Alice to Bob and an Alice-pointer from Bob to Alice.

1. ■ *Labeled pointing to friend.* Alice, who has no Dave-pointer to their friend Bob, may add one, if Bob has a Dave-pointer.
2. ■ *Labeled transitive pointing.* Alice, who knows Carol and has a Dave-pointer to Bob or is Bob's friend, may add a Dave-pointer to Carol, if Bob has a Dave-pointer to Carol.
3. ■ *Ceasing to point.* Alice, who has a Dave-pointer to Bob, may remove it.

Pragmas

Pointing: the persons Alice has pointers to, each with the labels of the pointers.
Labeled pointing to friend: `Alice [Follow] <label>`
Ceasing to point: `Alice [Unfollow]`

A label reaches Alice from a person Alice points to. Friends have each other's pointers, labeled by the friend: befriending records them and ending a friendship removes them (GSG:1, GSG:2), so GDO-L replaces the two schemas of GSG, which Appendix B shows. A person who followed a feed through its author before befriending them holds the pointer twice, and each act removes one. By GDO-L:1 Alice adds a Dave-pointer to their friend Bob, where Bob has one; by GDO-L:2 Alice, who has a Dave-pointer to Bob or is Bob's friend, adds one to Carol, whom Alice knows and to whom Bob has one. As in GDO, adding a pointer is an act of the person who points and is recorded in their state, and the person pointed to keeps no record of it.

GFF: Grassroots Feeds and Followers

GFF extends GDO-L.
Alice, Bob and Carol are persons, Bob and Carol possibly one person. A feed is named by its author, and Alice follows Carol's feed through Bob when Alice has a Carol-pointer to Bob.

1. ■ *Feed.* Alice has a feed and may post to it an item, created or received.
2. ■ *Delivery.* Alice, who has a Carol-pointer to Bob, receives from Bob every item of Carol's feed that Bob holds.

Pragmas

Posts: the items Alice holds.
Feeds: the feeds Alice follows; under a feed, the persons Alice follows it through and its items, each once.
Feed: `Alice [Post] <item>`

The acts a person wills are posting an item (GFF:1) and the three acts of GDO-L; delivery (GFF:2) is the machines'. Alice follows the feed of their friend Carol by the Carol-pointer friends have, and the feed of any other person through a friend who follows it (GDO-L:1). Alice may then follow it through the person their friend follows it through, if Alice knows them (GDO-L:2), and so on, towards the author. An item is signed by its author (GKK:1), and Alice holds each item of a feed once, with the person they received it from. Alice may follow a feed through several persons, and then receives each of its items from one of them. An author keeps no record of their followers.

Pragmas. The pragmas of a contract end its box [17]: the app's tabs, and a button for each act a person wills, with the fields they fill [24]. The contracts are nondeterministic: GDO-L:1 and GDO-L:2 say which pointers Alice may add, and not which Alice adds. The restrictions of Sections 4 and 5 add acts no person wills that add pointers.

3.2 Speech-Act Schemas

The clauses of a contract describe acts, and its speech-act schemas [17] define them. GFF's two are below, as an example; those of GDO-L and of the restrictions are in Appendix B. GFF has the schemas of GKK and GDO, GSG's two as GDO-L has them, four of GDO-L and two of its own, over three predicates: $pointer(\text{Bob}, \text{Dave})$ in the state of Alice says that Alice has a Dave-pointer to Bob; $item(x, \text{Carol})$ that Alice holds the item x of Carol's feed; and $received(x, \text{Carol}, \text{Bob})$ that Alice received it from Bob. A label is a person, an agent variable in a schema. An item is a speech act, content signed by its author [18]. A schema is written with its head before the arrow and its body after. The first argument of an atom is its holder; the body is what the act requires of the machines and, negated, forbids; the head is what it adds and, negated, deletes; and the guard names the persons who must will it. Every role requires *identity*, which is not shown.

GFF: Speech-Act Schemas

1. $item(\text{Alice}, x, \text{Alice}) \leftarrow \neg item(\text{Alice}, x, \text{Alice})$ guarded by Alice.
2. $item(\text{Alice}, x, \text{Carol}), received(\text{Alice}, x, \text{Carol}, \text{Bob}) \leftarrow pointer(\text{Alice}, \text{Bob}, \text{Carol}), \neg item(\text{Alice}, x, \text{Carol}), item(\text{Bob}, x, \text{Carol})$.

GFF:1 is *post*; its item occurs in no atom the act requires, so it is a speech act Alice signs [17]. GFF:2 is *deliver*, which no person wills and which reads Bob's state.

Each schema denotes volition-guarded transactions among agents, an agent being a person with a will and a machine with a state, and the contract denotes the protocol over them [17, 30], which Appendix A recalls. A run of it is *correct* if it is safe and live: it takes only transactions of the protocol, and every class of transactions that stays enabled is eventually taken. So while Alice has a Carol-pointer to Bob, and Bob holds an item of Carol's feed that Alice does not hold, delivery of the item is enabled, and Alice comes to hold it, from Bob or from another person Alice follows the feed through.

3.3 Properties

The *feeds-and-followers protocol* is the protocol realising GFF [17]. A person p *posts* x by an act of GFF:1, after which p holds x as an item of the feed of p. The proofs are in Appendix D.

Proposition 3.1 (Provenance of a Feed). *In every run of the feeds-and-followers protocol, a person a holds an item x of the feed of l only if l posted x and signed it; if $a \neq l$, then a received x from a person who held it as an item of the feed of l and to whom a had an l-pointer; and a holds x once.*

Proposition 3.2 (Delivery). *In every correct run of the feeds-and-followers protocol, if from some configuration on a has an l-pointer to b and b holds an item x of the feed of l, then a comes to hold x. Hence if l posted x and $l = p_0, p_1, \ldots, p_n = a$ are persons such that, from the posting on, p_i has an l-pointer to p_{i-1} for every $1 \leq i \leq n$, then a comes to hold x.*

Proposition 3.3 (Grassroots Property). *The feeds-and-followers protocol is volitionally grassroots.*

The contract is syntactically grassroots (Proposition C.1), so the proposition follows from the theorem of [17] that the protocol realising such a contract is volitionally grassroots (Definition A.24).

3.4 An Implementable Restriction

The compiler of [24] compiles a contract only if, in each schema with no guarding role, the atoms its first role requires name every other role and every variable of the atoms it adds and deletes: the machine of that role's person starts the act, and finds it in its own state. We call such a contract *implementable*. GFF is not: in delivery, Alice's state names Bob and the feed and not the item, and Bob's names the item and not Alice. GFF-I is an implementable restriction of GFF: it has the schemas of GFF, three of them with stronger bodies and the same heads, guards and names [17, Section Relations among Contracts].

GFF-I: Implementable GFF

GFF-I restricts GFF.
Alice, Bob and Carol are persons, Bob and Carol possibly one person.

1. ■ *Labeled transitive pointing.* GDO-L:2, only if Carol knows Alice.
2. ■ *Delivery.* GFF:2, only if Bob knows Alice.

In delivery, Bob, its first role, requires that Bob knows Alice and holds the item, so Bob's state names Alice, the feed and the item; GFF-I's schemas are in Appendix B. A person who has a pointer to Bob is Bob's friend or is known to Bob by GFF-I:1, so Bob knows everyone who has a pointer to Bob, and a delivery GFF allows GFF-I allows too. GFF-I forbids Alice to add a pointer to a Carol who does not know Alice.

Proposition 3.4 (Certification of GFF-I). *GFF-I is syntactically grassroots and implementable.*

A contract C' *restricts* C if every correct run of C' is, on the machines, one of C: its machine configurations are included, in order, in those of a correct run of C, a run of C' possibly taking in one act what C takes in several [17, Definition Implementation, restriction]. Whether one contract restricts another is undecidable, and syntactic conditions approximate it, a set that is dynamic and that we hope and expect to grow [17, Proposition Syntactic restrictions]. GFF-I restricts GFF by two of them: strengthening the body of an act a person wills, adding a reading role, and strengthening the body of an act no person wills, under an invariant of the restriction, which Lemma D.1 states.

Proposition 3.5 (Restriction). *GFF-I restricts GFF.*

Propositions 3.1 and 3.2 hold of the protocol realising GFF-I, the one the app runs, with the same proofs, Lemma D.1 giving in the second that Bob knows Alice. The proofs are in Appendix D.

3.5 The App

[TBD: To be rewritten for GFF-I, from which the platform is compiled, once it is compiled.]

The transactions of Section 3.2 are implemented by vGLP's canonical platform of communicating volitional agents, which carries out each transaction by a two-phase commit coordinated by its initiator, compiled into a program in volition-guarded GLP (vGLP) [24]. vGLP extends GLP [48] with volitional procedures, whose goals put a typed question to the person and reduce on the answer. The implementation is correct by vGLP's theorem for its canonical platform [24]: Propositions 3.1 and 3.2's first part hold in every run of it, and a delivery is carried out in every run that meets the theorem's conditions.

In the grassroots app. The platform runs as a mini-app of the grassroots super-app [13], and Figure 1 shows it on the iPhone simulator in the play below, one person's screen each. Under each agent of the Feeds tab the app lists what the person's state says of them: the friends they follow the agent through, the friends whose updates say they follow the agent, and the items received, each once, with the signatures of each of its paths. Every word on a screen is the contract's, and a friend is named once for each atom naming them, so a friend Alice follows Bob through, whose update says so, is named twice.

A play of five agents. To show that the acts compose into a feed, we run a play of five agents, Alice, Bob, Carol, Dave and Eve, with the friendships Alice–Bob, Bob–Carol, Carol–Dave and Dave–Eve, each person scripted, as the record `cascade/GFF` shows:

(1) Each establishes their identity, and the four pairs of friends come to know each other and befriend (GKK:1, GKK:2, GSG:1); friends follow each other (GFF:**??**).
(2) Alice posts two items (GFF:1); Bob, her friend, follows her (GFF:**??**), and her machine delivers the two to him (GFF:2); his machine updates Carol that he follows Alice (GFF:**??**).
(3) Bob introduces Alice to Carol (GKK:3); Carol wants to follow Alice (GFF:**??**), and her machine follows Alice through Bob, whose update said he follows Alice (GFF:**??**); his machine forwards the two items to her, signed over by him. Dave does the same through Carol, and Eve through Dave.
(4) Alice posts a third item, which reaches Eve signed by Alice and over by Bob, Carol and Dave, in that order.
(5) Carol ceases to want to follow Alice (GFF:**??**), and her machine ceases following Alice through Bob (GFF:**??**); Dave's following through Carol ends, and Eve's through Dave, and Carol's machine updates Dave that she no longer follows Alice; he still wants to follow her, holds his three items, and no friend of his follows her, so his machine follows her through no one, and stays so.

A second play, of choice: Eve, told by both Bob and Frank that they follow Alice, presses Follow; her machine follows Alice through Frank, the first friend known to follow her, and through no other; the item reaches her along Alice, Carol, Dave, Frank; she presses Unfollow and her machine ceases. All five agents run concurrently as GLP processes, communicating over channels [48]; the play and the scripted persons are `programs/apps/gff` in the repository.

4 PATH SHORTENING

A follower of Dave's feed through Bob receives its items along a sequence of persons, each with a Dave-pointer to the next, the author last. By GFF-I:1 the follower may add a pointer to the next person but one, Carol, and by GDO-L:3 remove the pointer to Bob. GFF-PS restricts GFF-I with one act no person wills, which takes the two in one. The text in the box is the repository's copy, `programs/gff/GFF-PS.txt`.

GFF-PS: Path-Shortening GFF

GFF-PS restricts GFF-I.
Alice, Bob, Carol and Dave are persons, Carol and Dave possibly one person.

1. ■ *Shortening.* Alice, who knows Carol and has a Dave-pointer to Bob and none to Carol, while Bob has a Dave-pointer to Carol and Carol knows Alice, replaces it by a Dave-pointer to Carol.

The act's speech-act schema is in Appendix B. Alice's first Dave-pointer is Alice's own act, by which Alice says that they want to follow Dave's feed. Repeated, the act brings Alice to a pointer to the author, provided Alice and each person on the way know each other; Alice comes to know Carol by an introduction, an act of Bob and Alice (GKK:3), and Carol comes to know Alice likewise.

Proposition 4.1 (Certification of GFF-PS). *GFF-PS is syntactically grassroots and implementable.*

Proposition 4.2 (Path shortening). *GFF-PS restricts GFF-I.*

The act of GFF-PS takes in one act what GFF-I takes in two, which the definition of restriction allows and none of the syntactic

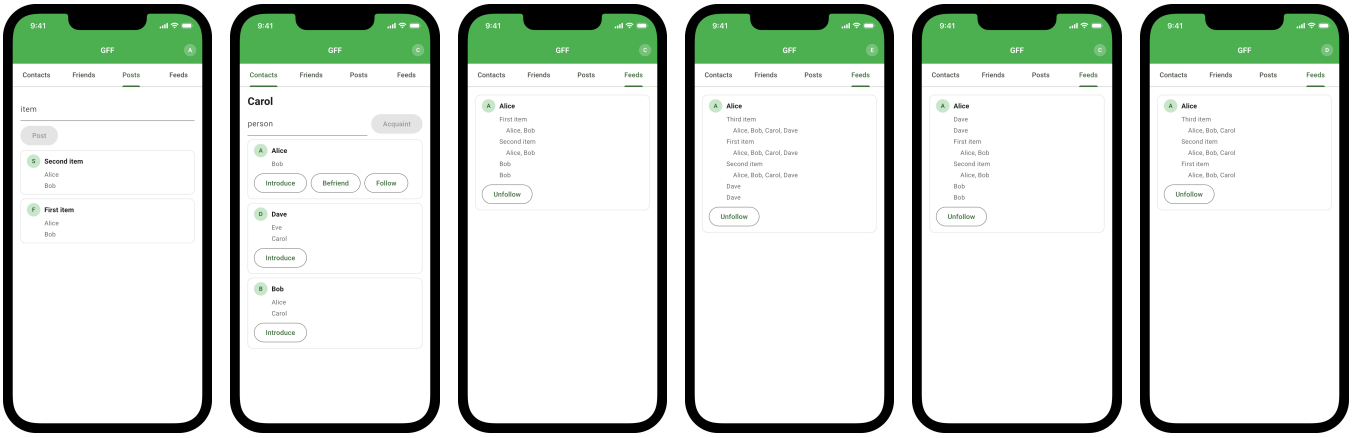


Figure 1: The GFF mini-app in the grassroots app, on the iPhone simulator, in the play of five agents, one person's screen each. (1) Alice's Posts after her two posts: Post with its field, and the two items, each with Alice, its signer, and Bob, the friend it was sent to (GFF:1, GFF:2). (2) Carol's Contacts after Bob introduced Alice to her: Alice's row with Bob, whose update says he follows her, and Introduce, Befriend and Follow (GFF:??). (3) Carol's Feeds after her machine followed Alice through Bob: Alice's row with the two items, each signed by Alice and Bob, then Bob twice, the friend she follows Alice through and his update, and Unfollow (GFF:??). (4) Eve's Feeds after the third item: Alice's row with the three items, each signed by Alice, Bob, Carol and Dave in order, Dave twice, and Unfollow. (5) Carol's Feeds after Dave followed Alice through her: Alice's row opening with Dave twice, he follows Alice through Carol, and his update. (6) Dave's Feeds after Carol's Unfollow: Alice's row remains, he still wanting to follow her, with the three items he holds and no friend under it, his following through Carol ended (GFF:??). This is a screenshot of the GFF mini-app, compiled from the GFF social contract down the abstraction cascade [17, 24]. One instance of the app on an iPhone simulator runs the mini-app of every agent; the agents communicate over channels [48] within it, and the screenshot shows one person's screen. All source code and all artefacts, including the social contracts and the artefacts the abstraction cascade produces from them, are in the public repository https://github.com/EShapiro2/GLP.

approximations of [17] covers, so the proof is direct. The proofs are in Appendix D.

4.1 Paths in a Random Social Graph

[TBD: The study below measured following through friends with forwarding and path shortening by the signatures of an item, in the contracts of 2026-10-28; it is to be run again on GFF-I, GFF-PS and GFF-FR.]

A follower receives an item through a path of friends, each of whom holds it, and the length of the path is the number of hands it passes through. How long the paths are, and whether a person who wishes to follow finds a friend who follows at all, depends on the social graph and on how many follow an author; so we measured them on a random social graph, by a script beside the contract, `programs/gff/paths` in the repository https://github.com/EShapiro2/GLP.

The graph and the scenario. A grassroots social graph is of friendships between people who know each other personally, so a person's degree is bounded, friends of a friend are often friends, and a few friendships reach far. We take Kleinberg's grid [26]: 100×100 persons, each a friend of their four grid neighbours and of q others, drawn with probability falling as the square of the grid distance, q in 1, 2 and 4, five seeds each. Every person is an author, with a number of followers drawn from a power law of exponent 2, so a few authors have thousands of followers and most a handful; the followers of an author are persons drawn at random; and the followers of all authors come to follow in one random order. A person who wants to follow an author follows through the friend with the shortest path, the author's friends following by friendship from the start; one none of whose friends follows waits, and is tried again whenever a friend comes to follow. After all have followed, shortening runs until no follower has a friend with a shorter path. Under GFF-PS alone, by the signatures of the items received, no follower switched in any run: every signer of an item a follower receives followed the author before that follower chose, and a signer who is a friend was a candidate then; the clause acts when a friendship or a following changes after the choice, as in the play of Section 3.5. The shortening measured is therefore the full one, a follower switching to any friend with a shorter path. We measured, over all author–follower pairs and by the author's number of followers, the fraction waiting, the distribution of the path length, and its ratio to the follower's distance from the author in the graph, which is the best a path can be.

Table 1: Paths on Kleinberg's grid with $q = 2$, five seeds pooled, by the author's number of followers: those who follow and the fraction waiting; the mean and the greatest path length at the first following, after shortening, and the graph distance; and the mean ratio of path length to graph distance.

followers	following	waiting	first	max	shortened	max	distance
1–9	505	0.994	1.87	3	1.87	3	1.87
10–99	444	0.994	1.89	4	1.89	4	1.89
100–999	494	0.993	2.22	7	2.22	7	2.18
1000–9999	29,415	0.357	26.7	84	8.41	27	5.42

Results. Table 1 shows the runs with $q = 2$; those with $q = 1$ and $q = 4$ differ in degree and not in kind. With up to a thousand followers, 99 per cent of those who wish to follow wait for ever: none of their friends follows the author, and those who do follow are the author's friends and their friends, at path length two. With thousands of followers, two thirds follow, at a first path five times their distance from the author, up to 84 hands; shortening brings the mean to 8.4, 1.6 times the distance, and the same as the tree of shortest paths over the followers, which is the best any following through friends can do. Reaching two and three hops deep, a friend following on the other's behalf, leaves 95 and 36 per cent of those who wish to follow an author with ten to a hundred followers waiting, and makes 60 to 98 per cent of the population hold feeds they never asked for. So following through friends serves large authors, after shortening, and not small ones; the follower of a small author, who knows the author, befriends them (GSG:1) and follows as a friend, which is the Friendship pragma.

5 FAULT-RESILIENT FEEDS

A follower who follows Carol's feed through one person receives no item of it while that person is down. GDO-L lets Alice have Carol-pointers to several persons, and Alice then receives each item from one of them who holds it (Proposition 3.2). GFF-FR restricts GFF-I with one act no person wills, which adds such pointers up to a bound. The text in the box is the repository's copy, `programs/gff/GFF-FR.txt`.

GFF-FR: Fault-Resilient GFF

GFF-FR restricts GFF-I.
Alice, Bob and Carol are persons. $d > 1$ is an integer.

1. ■ *Redundant following.* Alice, who has Carol-pointers to at least one person and to fewer than d, adds a Carol-pointer to their friend Bob, who has a Carol-pointer.

The act's speech-act schema is in Appendix B. Alice's first Carol-pointer is Alice's own act, or their friendship with Carol. A person who was down receives, when up, the items they missed from the persons they have pointers to, by GFF:2, so no person forwards an item to another.

Proposition 5.1 (Certification of GFF-FR). *GFF-FR is syntactically grassroots and implementable.*

Proposition 5.2 (Fault resilience). *GFF-FR restricts GFF-I.*

The proofs are in Appendix D.

5.1 Back-Channels

A follower who follows Dave's feed through Bob and Carol, and receives an item first from Bob, holds it while Carol may not, Carol's own path being down or slower. GFF-FR-BC extends GFF-FR with a back-channel, by which the follower forwards the item to Carol. The text in the box is the repository's copy, `programs/gff/GFF-FR-BC.txt`.

GFF-FR-BC: Fault-Resilient GFF with Back-Channels

GFF-FR-BC extends GFF-FR.
Alice, Bob, Carol and Dave are persons, Bob and Dave possibly one person.

1. ■ *Back-channel.* Alice, who has Dave-pointers to Bob and to Carol and receives an item of Dave's feed first from Bob, forwards it to Carol.

The act's speech-act schema is in Appendix B; Carol records the item and its sender with *received_back*. Alice holds each item once, with the person she received it from, so "first from Bob" needs no clock. A back-channel item exposes only its sender: Carol learns that Alice has a Dave-pointer to Carol. If Alice is Carol's friend, Carol may add a pointer to the person Alice follows the feed through (GDO-L:2), which may be nearer Dave than Carol's own.

Proposition 5.3 (Certification of GFF-FR-BC). *GFF-FR-BC is syntactically grassroots and implementable.*

Proposition 5.4 (Back-channel). *In every correct run of the protocol realising GFF-FR-BC, if from some configuration on a has l-pointers to b and to c and holds an item x of the feed of l received from b, then c comes to hold x. In every run of it, a person a holds an item x of the feed of l only if l posted x and signed it; if $a \neq l$, then a received x from a person to whom a had an l-pointer, or who had an l-pointer to a; and a holds x once.*

The proofs are in Appendix D.

[TBD: The measurements below are of the contracts of 2026-10-28, following through friends with forwarding, and their extensions for shortening and redundancy; they are to be run again on GFF-I, GFF-PS, GFF-FR and GFF-FR-BC.]

Measured. On the grid of Section 4.1 with time — friendships and followings arriving in a random order, each author posting at a steady rate, a message one step per hand — GFF, GFF-PS, GFF-FR and the two together were run on the same seeds and authors, five seeds, $q \in \{1, 2, 4\}$, an item never sent to one who signed it, and the study's choice where the contracts leave one: an item that comes by redundant forwarding switches nothing, and where the friend who signed it is already the other following, the following the item came by ends and a second is taken through another friend. For authors with thousands of followers, without faults, an item passed through 49.2 hands under GFF with $q = 1$, 44.5 under GFF-PS, 15.1 under GFF-FR and 13.9 under both, 14.4 and 13.5 with the two friends chosen far apart, by the fewest agents in common on their paths; with $q = 2$, 37.8, 35.0, 14.5 and 13.8; with $q = 4$, 31.4, 28.9, 10.5 and 9.8; at one message per follower reached under GFF and GFF-PS and about two under redundancy, and tens of thousands of switches for the last half-hand. With faults, each agent going down at a small rate for a random stretch, about one agent in ten down at any time, GFF reached 97.8 per cent of an item's followers by the end of the run with $q \in \{1, 2\}$ and 99.8 with $q = 4$, GFF-PS 98.5, 98.2 and 99.7, and GFF-FR, with or without shortening, every follower; an item took 2,650 to 3,900 steps on the mean to reach a follower under GFF, 2,450 to 3,700 under GFF-PS, and 175 to 227 under redundancy. With one agent in a hundred down every contract reached every follower, GFF in 334 to 447 steps and redundancy in 29 to 35. Redundancy does the work; shortening on top of it is worth half a hand to one.

6 RELATED WORK

6.1 Feeds and Followers in Decentralised Social Networks

A feed with one author and many followers is realised, in the decentralised social networks of the last two decades, by a feed held by a third party or by anyone, a follow the follower records alone, and delivery through a global index, a replicated state, or servers each side chooses. We take the systems in turn, by where the feed is held, how a follow is recorded, and how a post reaches a follower.

Bluesky and the AT Protocol. Each account has one repository, a signed Merkle search tree of its records, posts and follows among them, held by the account's personal data server, which holds the signing key too [27]. A follow is a record in the follower's repository; the author does not consent and learns of it only by indexing, and the follow is public. A relay aggregates the commit streams of many servers into one firehose, an application view indexes the firehose, keeps each person's followers and builds their timeline, and the reader's server fetches it from the view: delivery goes through an index of the whole network, and two views over different relays index different networks [27].

ActivityPub and Mastodon. An actor's outbox, a collection in reverse chronological order, is held by the actor's server [38]. A follow is a Follow activity delivered to the author's inbox, which the author's server accepts or rejects, and both servers keep a collection, of followers and of following. The server pushes each post to the inbox of each follower's server, one hop along the follow graph [38]; Mastodon's instances hold the accounts, and the top five per cent of them hold ninety per cent of the accounts [42].

Nostr and Farcaster. In Nostr an event is signed by its author and stored by relays, with no chain between events; a follow list is an event the follower publishes, and a follower reads an author's events by a standing query at a relay both use [34]. In Farcaster each message is signed by a key registered on a blockchain, held by every hub, which gossip every message to every hub; a follow is a link message of the follower, public and replicated everywhere, and an application computes a feed from a hub's whole state [15].

Secure Scuttlebutt. A feed is the signed append-only log of one identity, and a follow a contact message in the follower's own log, taken without the author's consent and public; peers that connect replicate the logs each is interested in, up to a set number of hops in the follow graph, so an author's log reaches a follower through peers who need not follow the author, and is held by them [25, 53]. Scuttlebutt is the closest of these to GFF in spirit: no server, a signed log per author, and replication between peers. GFF differs in that a follower receives an item from a person they point to with the feed, its author or another follower, an item is held by the author and the followers, and an author keeps no record of their followers; a follower holds nothing of a feed they do not follow.

Logs, chains and swarms. Dat and Hypercore replicate a signed single-writer log through a swarm found by a distributed hash table, with the key as the capability to read and no follow at all [36]; DSNP publishes announcements to a blockchain, from which an application reads the whole stream, with public follows, encrypted follows and two-sided private connections in the follower's data [39]; and peer-to-peer and blockchain-based social networks [5, 21, 23, 41] hold the feed in a distributed hash table or a chain. In every one of these the feed is held by agents other than the author and the followers, and the follow graph is public or held by a server.

Grassroots social networking. The Twitter-like protocol of [46] has personal feeds with authors and followers, the feed the author's signed blocks in a blocklace, a follow a block the follower creates, and dissemination along the edges of the social graph only, a block travelling to a friend who follows its author; a follower who is not the author's friend receives the feed through a path of friends who all follow the author. GFF is that protocol written as a grassroots contract over a labeled directed overlay: following a feed is pointing with it to a friend who follows it, so the path exists by construction, a follower holds each item once, and the implementation is compiled from the contract.

Dissemination. Publish/subscribe decouples publishers from subscribers through an event service that stores the subscriptions and routes the events, centralised, through brokers, or in the participants' own processes [14]; a topic per author is a feed, and the service, not the author, knows the subscribers. Epidemic dissemination spreads an update from one site to all by direct mail, anti-entropy or rumour mongering [12]; Scuttlebutt's replication and Farcaster's synchronisation are anti-entropy. GFF's delivery is neither: an item passes from the author to the persons who point to the author with the feed, and on to those who point to them, and the author keeps no record of followers.

Centralised timelines. A centrally-controlled social network holds the follow graph, the posts and each person's timeline, built by fan-out on write into each follower's cache, or merged at read time for the largest accounts; RSS and Atom are a feed document at the publisher's address, pulled by each reader, with no subscriber list [35].

Formal foundations and other platforms. Atomic transactions were investigated early in distributed computing [28, 31, 32], for their implementation [4, 8], in programming languages [3] and in process calculi [1, 10, 11]; the framework here builds on multiagent transition systems [43] with atomic transactions [50] and volition guards [30]. Digital social contracts [6] are the digital incarnation of Proudhon's [40], realised as grassroots platforms [45, 50], among them social networks [46], currencies [29, 44] and federations [22, 52]. Contract languages — contract automata [2, 7], a loan as an automaton [16], L4 [54], defeasible deontic rules [19, 20], Symboleo [37, 51], Stipula [9] and Catala [33] — fix the parties when the contract is formed, where speech-act schemas [17] quantify over every finite set of people.

7 CONCLUSION

[TBD: The conclusion follows the paper.]

AI Disclosure. The authors used Anthropic's Claude models throughout, under their direction and review, and made all design decisions. The clauses of the contract are the authors', drafted with Claude. The prose was drafted with Claude. The authors verified the correctness and originality of all content, including all references, and responsibility for all content rests with them.

REFERENCES

[1] Lucia Acciai, Michele Boreale, and Silvano Dal Zilio. 2007. A concurrent calculus with atomic transactions. In *European Symposium on Programming*. Springer, 48–63.

[2] Shaun Azzopardi, Gordon J. Pace, Fernando Schapachnik, and Gerardo Schneider. 2016. Contract automata. *Artificial Intelligence and Law* 24, 3 (2016), 203–243. https://doi.org/10.1007/s10506-016-9185-2

[3] Johannes Borgström, Karthikeyan Bhargavan, and Andrew D Gordon. 2009. A compositional theory for STM Haskell. In *Proceedings of the 2nd ACM SIGPLAN Symposium on Haskell*. 69–80.

[4] Manuel Bravo and Alexey Gotsman. 2019. Reconfigurable atomic transaction commit. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*. 399–408.

[5] Sonja Buchegger, Doris Schiöberg, Le-Hung Vu, and Anwitaman Datta. 2009. PeerSoN: P2P social networking: early experiences and insights. In *Proceedings of the Second ACM EuroSys Workshop on Social Network Systems*. 46–52. https://doi.org/10.1145/1578002.1578010

[6] Luca Cardelli et al. 2020. Digital social contracts: A foundation for an egalitarian and just digital society. In *CEUR Proceedings of the First International Forum on Digital and Democracy*, Vol. 2781. CEUR-WS, 51–60. https://ceur-ws.org/Vol-2781/paper5.pdf

[7] Stefan Chircop, Gordon J. Pace, and Gerardo Schneider. 2022. An Automata-Based Formalism for Normative Documents with Real-Time. In *Legal Knowledge and Information Systems: JURIX 2022 (Frontiers in Artificial Intelligence and Applications, Vol. 362)*. IOS Press, 158–163. https://doi.org/10.3233/FAIA220460

[8] Gregory Chockler and Alexey Gotsman. 2021. Multi-shot distributed transaction commit. *Distributed Computing* 34 (2021), 301–318.

[9] Silvia Crafa, Cosimo Laneve, and Giovanni Sartor. 2022. Pacta sunt servanda: Legal Contracts in Stipula. *Science of Computer Programming* (2022). https://arxiv.org/abs/2110.11069 arXiv:2110.11069.

[10] Edsko de Vries, Vasileios Koutavas, and Matthew Hennessy. 2010. Communicating transactions. In *International Conference on Concurrency Theory*. Springer, 569–583.

[11] Edsko De Vries, Vasileios Koutavas, and Matthew Hennessy. 2010. Liveness of communicating transactions. In *Asian Symposium on Programming Languages and Systems*. Springer, 392–407.

[12] Alan Demers et al. 1987. Epidemic algorithms for replicated database maintenance. In *Proceedings of the Sixth Annual ACM Symposium on Principles of Distributed Computing (PODC '87)*. 1–12.

[13] Ohad Eitan, Idit Keidar, and Ehud Shapiro. 2026. The Grassroots Super-App: A Secure Operating System for Grassroots Mini-Apps. arXiv:XXX:XXXXX. arXiv:XXX:XXXXX [cs.DC]

[14] Patrick Th. Eugster, Pascal A. Felber, Rachid Guerraoui, and Anne-Marie Kermarrec. 2003. The many faces of publish/subscribe. *Comput. Surveys* 35, 2 (2003), 114–131.

[15] Farcaster. 2024. Farcaster Protocol Specification. https://github.com/farcasterxyz/protocol. Accessed 2026.

[16] Mark D. Flood and Oliver R. Goodenough. 2022. Contract as automaton: representing a simple financial agreement in computational form. *Artificial Intelligence and Law* 30, 3 (2022), 391–416. https://doi.org/10.1007/s10506-021-09300-9

[17] James Golike, Andy Lewis-Pye, and Ehud Shapiro. 2026. Grassroots Social Contracts: From Legal Text to an App (Full Version). *arXiv:2609.06231* (2026).

[18] James Golike and Ehud Shapiro. 2026. Digital Speech Acts Retain Control of Copyright with People, Not Platforms. *arXiv preprint arXiv:2606.19263* (2026). https://arxiv.org/abs/2606.19263

[19] Guido Governatori et al. 2018. On legal contracts, imperative and declarative smart contracts, and blockchain systems. *Artificial Intelligence and Law* 26, 4 (2018), 377–409. https://doi.org/10.1007/s10506-018-9223-3

[20] Guido Governatori and Antonino Rotolo. 2004. Modelling Contracts Using RuleML. In *Legal Knowledge and Information Systems: JURIX 2004 (Frontiers in Artificial Intelligence and Applications)*, Thomas F. Gordon (Ed.). IOS Press, 141–150.

[21] Barbara Guidi, Marco Conti, Andrea Passarella, and Laura Ricci. 2018. Managing social contents in decentralized online social networks: a survey. *Online Social Networks and Media* 7 (2018), 12–29. https://doi.org/10.1016/j.osnem.2018.07.001

[22] Daniel Halpern, Ariel D Procaccia, Ehud Shapiro, and Nimrod Talmon. 2024. Federated Assemblies. *Proc AAAI 2025; arXiv preprint arXiv:2405.19129* (2024). https://arxiv.org/abs/2405.19129

[23] Le Jiang and Xinglin Zhang. 2019. BCOSN: A blockchain-based decentralized online social network. *IEEE Transactions on Computational Social Systems* 6, 6 (2019), 1454–1466.

[24] Idit Keidar and Ehud Shapiro. 2026. Volition Elicitation: Operational Semantics for People and Their Machines. arXiv:2607.14138. arXiv:2607.14138 [cs.DC] https://arxiv.org/abs/2607.14138

[25] Anne-Marie Kermarrec, Erick Lavoie, and Christian Tschudin. 2020. Gossiping with append-only logs in secure-Scuttlebutt. In *Proceedings of the 1st international workshop on distributed infrastructure for common good*. 19–24. https://doi.org/10.1145/3428662.3428794

[26] Jon Kleinberg. 2000. The small-world phenomenon: an algorithmic perspective. In *Proceedings of the Thirty-Second Annual ACM Symposium on Theory of Computing (STOC '00)*. 163–170.

[27] Martin Kleppmann et al. 2024. Bluesky and the AT protocol: Usable decentralized social media. *arXiv preprint arXiv:2402.03239* (2024). https://arxiv.org/abs/2402.03239

[28] Butler W Lampson. 1981. Chapter 11. atomic transactions. In *Distributed Systems—Architecture and Implementation: an Advanced Course*. Springer, 246–265.

[29] Andrew Lewis-Pye, Oded Naor, and Ehud Shapiro. 2023. Grassroots Flash: A Payment System for Grassroots Cryptocurrencies. *arXiv preprint arXiv:2309.13191* (2023). https://arxiv.org/abs/2309.13191

[30] Andy Lewis-Pye and Ehud Shapiro. 2026. Volition-Guarded Multiagent Atomic Transactions: Describing People and their Machines. *arXiv preprint arXiv:2604.25596* (2026). https://arxiv.org/abs/2604.25596

[31] Nancy Lynch, Michael Merritt, William Weihl, and Alan Fekete. 1988. A theory of atomic transactions. In *ICDT'88: 2nd International Conference on Database Theory Bruges, Belgium, August 31–September 2, 1988 Proceedings 2*. Springer, 41–71.

[32] Nancy A Lynch and Michael Merritt. 1993. *Atomic transactions: in concurrent and distributed systems*. Morgan Kaufmann Publishers Inc.

[33] Denis Merigoux, Nicolas Chataing, and Jonathan Protzenko. 2021. Catala: A Programming Language for the Law. *Proceedings of the ACM on Programming Languages* 5, ICFP (2021). https://doi.org/10.1145/3473582

[34] Nostr Protocol. 2024. NIP-01: Basic Protocol Flow Description. https://github.com/nostr-protocol/nips/blob/master/01.md. Accessed 2026.

[35] Mark Nottingham and Robert Sayre. 2005. The Atom Syndication Format. RFC 4287, Internet Engineering Task Force.

[36] Maxwell Ogden, Karissa McKelvey, and Mathias Buus. 2017. Dat - Distributed Dataset Synchronization And Versioning. https://github.com/datprotocol/docs. White paper, version 2017-01-17.

[37] Alireza Parvizimosaed et al. 2022. Specification and Analysis of Legal Contracts with Symboleo. *Software and Systems Modeling* 21 (2022). https://doi.org/10.1007/s10270-022-01053-6

[38] Evan Prodromou et al. 2018. ActivityPub. https://www.w3.org/TR/activitypub/. https://www.w3.org/TR/activitypub/ W3C Recommendation.

[39] Project Liberty. 2021. Decentralized Social Networking Protocol (DSNP) Specification. https://dsnp.org/. Version 1.0.

[40] Pierre-Joseph Proudhon and John Beverley Robinson. 2004. *General idea of the revolution in the nineteenth century*. Courier Corporation. Original work published 1851.

[41] Mohsin Ur Rahman, Barbara Guidi, and Fabrizio Baiardi. 2020. Blockchain-based access control management for decentralized online social networks. *J. Parallel and Distrib. Comput.* 144 (2020), 41–54.

[42] Aravindh Raman et al. 2019. Challenges in the decentralised web: The mastodon case. In *Proceedings of the internet measurement conference*. 217–229. https://doi.org/10.1145/3355369.3355572

[43] Ehud Shapiro. 2021. Multiagent Transition Systems for Composing Fault-Resilient Protocol Stacks. *arXiv preprint arXiv:2112.13650* (2021). https://arxiv.org/abs/2112.13650

[44] Ehud Shapiro. 2022. Grassroots Currencies: Turning Mutual Trust into Liquidity. *arXiv preprint arXiv:2202.05619* (2022). https://arxiv.org/abs/2202.05619

[45] Ehud Shapiro. 2023. Grassroots Distributed Systems: Concept, Examples, Implementation and Applications (Brief Announcement). In *37th International Symposium on Distributed Computing (DISC 2023). (Extended version: arXiv:2301.04391)*. LIPICS, Italy, 47:1–47:7. https://arxiv.org/abs/2301.04391

[46] Ehud Shapiro. 2023. Grassroots Social Networking: Serverless, Permissionless Protocols for Twitter/LinkedIn/WhatsApp. In *OASIS '23* (Rome, Italy). Association for Computing Machinery. https://doi.org/10.1145/3599696.3612898

[47] Ehud Shapiro. 2024. A Grassroots Architecture to Supplant Global Digital Platforms by a Global Digital Democracy. *arXiv:2404.13468, Proceedings of DAWO'24* (2024). https://arxiv.org/abs/2404.13468

[48] Ehud Shapiro. 2025. GLP: A Grassroots, Multiagent, Concurrent, Logic Programming Language for AI (full version). *arXiv preprint arXiv:2510.15747, Summary in Proc. of ICLP'26, EPTCS 450, pp. 119–133* (2025). https://arxiv.org/abs/2510.15747

[49] Ehud Shapiro. 2026. Child-Safe Social Networking: From a Grassroots Social Contract to an App. *In preparation* (2026).

[50] Ehud Shapiro. 2026. Grassroots Platforms with Atomic Transactions: Social Graphs, Cryptocurrencies, and Democratic Federations. In *Proceedings of the 27th International Conference on Distributed Computing and Networking*. 71–81. https://doi.org/10.1145/3772290.3772309 arXiv preprint arXiv:2502.11299.

[51] Sepehr Sharifi et al. 2020. Symboleo: Towards a Specification Language for Legal Contracts. In *IEEE 28th International Requirements Engineering Conference (RE)*. IEEE, 364–369. https://doi.org/10.1109/RE48521.2020.00049

[52] Nimrod Talmon and Ehud Shapiro. 2025. Grassroots Federation: Fair Democratic Governance at Scale. *arXiv preprint arXiv:2505.02208; also Proc. of AAMAS'26* (2025). https://arxiv.org/abs/2505.02208

[53] Dominic Tarr, Erick Lavoie, Aljoscha Meyer, and Christian Tschudin. 2019. Secure scuttlebutt: An identity-centric protocol for subjective and decentralized applications. In *Proceedings of the 6th ACM conference on information-centric networking*. 1–11. https://doi.org/10.1145/3357150.3357396

[54] Seng Joe Watt, Oliver Goodenough, and Meng Weng Wong. 2023. Deontics and Time in Contracts: An Executable Semantics for the L4 DSL. In *Legal Knowledge and Information Systems: JURIX 2023 (Frontiers in Artificial Intelligence and Applications)*. IOS Press, 119–124. https://doi.org/10.3233/FAIA230954

A VOLITION-GUARDED MULTIAGENT ATOMIC TRANSACTIONS

This appendix recalls, in the words of [30], the mathematical foundations on which the paper is based; the proofs of the results recalled are in that paper. It leaves out the volition-guarded if-then-else transactions of [30], which the contracts of this paper do not use.

Each agent consists of a *person* and a *machine* (e.g., smartphone) operated by the person. A *machine transaction* specifies what machines do; a *volition guard* specifies which persons must be willing for the transaction to occur.

We assume a potentially infinite set of *agents* Π, but consider only finite subsets of it, so we assume any particular set of agents $P \subset \Pi$ to be nonempty and finite. We use $\subset$ to denote the strict subset relation and $\subseteq$ when equality is also possible. As standard, we use S^P to denote the set of all total functions from P to S, and if $c \in S^P$ we use c_p to denote the value of c at $p \in P$.

Definition A.1 (Machine State, Configuration, Transaction, Volition–Guarded Multiagent Atomic Transaction [30]). Given an arbitrary set S of *machine states*, with a designated *initial state* $s_0 \in S$, and agents $Q \subset \Pi$, a *machine configuration* over Q is a member of S^Q, and a *machine transaction* over *participants* Q is a pair $c \to c' \in (S^Q)^2$ such that $c \neq c'$; it is *unary* if $|Q| = 1$. Given such a machine transaction t, a *volition-guarded multiagent atomic transaction* over t—henceforth, *volition-guarded transaction*—is a pair (t, Q') where $Q' \subseteq Q$ are its *guards*.

When we say a transaction is "guarded by $\{p, q\}$," both must be willing; when we say it is "guarded by either p or q," we mean there are two volition-guarded transactions over the same machine transaction, $(t, \{p\})$ and $(t, \{q\})$, so that either agent's volition suffices. A volition-guarded transaction whose guard is empty ($Q' = \emptyset$) is *unguarded* and is enabled whenever its machine precondition holds.

Distinct machine transactions can represent "the same action" in different configurations; this is captured with an equivalence relation on machine transactions.

Definition A.2 (Transaction Equivalence [30]). Given a set of machine transactions R, a *transaction equivalence* is an equivalence relation $\sim$ on R such that $t \sim t'$ implies t and t' have the same participants, and $R/\sim$ is countable. We write $[t]$ for the equivalence class of t under $\sim$.

For example, all befriend(p, q) transactions—differing only in the configurations in which they occur—form an equivalence class. Next is the mathematical machinery allowing people to express their volitions regarding the classes of machine transactions their machines may participate in.

Definition A.3 (Agent State and Configuration [30]). Given agents P, states S with initial state s_0, a set of machine transactions T each over its own participants $Q \subseteq P$ and S, and equivalence $\sim$ on T, an *agent state* is a pair $(V, m) \in \mathcal{A} = (2^{T/\sim} \times S)$ where V is its *volitional state* and $m \in S$ its *machine state*. The *initial agent state* is $(\emptyset, s_0)$. An *agent configuration* c over P, S, T, and $\sim$ is a member $c \in \mathcal{A}^P$ in which $c_p^v \subseteq (T/\sim)_p$ for every $p \in P$, where $(T/\sim)_p$ denotes the classes in $T/\sim$ in which p is a participant; we write c_p^v for the volitional state and c_p^m for the machine state of agent p in c.

Definition A.4 (Volitional Transaction [30]). Given agents P, states S, machine transactions T over P and S, and equivalence $\sim$ on T:

(1) A *volition change of agent* $p \in P$ is a pair $c \to c'$ of agent configurations over $\{p\}$, S, T, and $\sim$ such that $c_p^v, {c'}_p^v \subseteq (T/\sim)_p$ and $c_p^v \neq {c'}_p^v$, and $c_p^m = {c'}_p^m$.
(2) A *volitional machine transaction* induced by a volition-guarded transaction (t, Q'), for some $t = (d \to d') \in T$ over $Q' \subseteq Q \subseteq P$, is a pair $c \to c'$ where $c \neq c'$ are agent configurations over P, S, T, and $\sim$ such that $[t] \in c_q^v$ for every $q \in Q'$; $c_p^m = d_p$ and ${c'}_p^m = d'_p$ for every $p \in Q$; $c_p^m = {c'}_p^m$ for every $p \in P \setminus Q$; and ${c'}_p^v = c_p^v \setminus \{[t]\}$ for every $p \in P$.
(3) A *volitional transaction* is a volition change or a volitional machine transaction.

When a volitional machine transaction induced by (t, Q') is taken, the class $[t]$ is removed from every agent's volitional state: the will is fulfilled by any equivalent transaction. A person may independently change their volitional state via volition changes, which may add or remove classes; beyond these, the framework removes a class from c_p^v only upon fulfilment.

Induced transition systems. The following is a simplified variation, sufficient for the purpose of this work, on the foundations introduced in [43].

Definition A.5 (Transition System, Computation, Run, Safe, Live, Correct [30]). A *transition system* is a tuple $TS = (S, s_0, T, \sim)$, where:

(1) S is an arbitrary non-empty set, referred to as the set of *states*.
(2) Some $s_0 \in S$ is the designated *initial state*.
(3) $T \subseteq S^2$ is a set of *correct transitions over* S, where each transition $t \in T$ is a pair (s, s') of non-identical states $s \neq s' \in S$, also written as $t = s \to s'$.
(4) $\sim$ is a *partial equivalence relation* on T: a symmetric and transitive relation on T, not necessarily reflexive. Its domain $\{t \in T : t \sim t\}$ is partitioned into countably many *liveness classes* $T/\sim$; a transition outside the domain belongs to no class.

A *computation* of TS is a (nonempty, potentially infinite) sequence of states $r = s_1, s_2, \cdots$; it is a *run* of TS if $s_1 = s_0$. A *prefix* of a computation is a finite initial segment of it. A computation $r = s_1, s_2, \ldots$ is *safe*, also written $r \subseteq T$, if $s_i \to s_{i+1} \in T$ for every two consecutive states; and $s \xrightarrow{*} s' \subseteq T$ denotes the existence of a safe computation from s to s' (empty if $s = s'$). A class $[t] \in T/\sim$ is *enabled* in a state s if $s \to s' \in [t]$ for some $s' \in S$. A run r is *live* if no class $[t] \in T/\sim$ is enabled in every state of some suffix of r with no member of $[t]$ occurring in the suffix. A run is *correct* if it is safe and live.

A partial equivalence, rather than a total one, is used so that some transitions may carry no liveness requirement at all: only transitions in the domain of $\sim$ form classes and thereby incur a liveness obligation, while transitions outside the domain—belonging to no class—may occur in a correct run but are never required to.

A run with uncountably many classes enabled in every state of a suffix is not live.

Lemma A.6 (Extension [30]). *Every finite safe run of a transition system is a prefix of a correct run of it.*

Definition A.7 (Multiagent Transition System [30]). Given agents $P \subset \Pi$ and an arbitrary set S of *states* with a designated *initial state* $s_0 \in S$, a *multiagent transition system* over P and S is a transition system $TS = (C, c_0, T, \sim)$ with *configurations* $C := S^P$, *initial configuration* $c_0 := \{s_0\}^P$, *transitions* $T \subseteq C^2$ a set of transactions over P and S, and $\sim$ a partial equivalence on T.

Rather than specifying a multiagent transition system over a set of agents P directly, it is specified via machine transactions (Definition A.1). A machine transaction over $Q \subseteq P$ defines a set of multiagent transitions over P in which all members of $P \setminus Q$ are stationary:

Definition A.8 (Transaction Closure [30]). Let $P \subset \Pi$, S a set of machine states, and $C := S^P$. For any transition or transaction $t = c \to c'$, we write $t_q := c_q \to c'_q$ and say p is *stationary* in t if $c_p = c'_p$. For a machine transaction $t = (c \to c')$ over S with participants Q, the *P-closure of t*, $t{\uparrow}P$, is the set of transitions over P and S defined by:

$$t{\uparrow}P := \begin{cases} \{t' \in C^2 : \forall q \in Q.(t_q = t'_q) \wedge \forall p \in P \setminus Q.(p \text{ is stationary in } t')\} & \text{if } Q \subseteq P \\ \emptyset & \text{otherwise} \end{cases}$$

If R is a set of machine transactions, each $t \in R$ over some Q and S, then the *P-closure of R*, $R{\uparrow}P$, is the set of transitions over P and S defined by:

$$R{\uparrow}P := \bigcup_{t \in R} t{\uparrow}P$$

Given a relation $\sim$ on R, its *P-closure* $\sim{\uparrow}P$ is the relation on $R{\uparrow}P$ with $\hat{t}$ $(\sim{\uparrow}P)$ $\hat{t}'$ iff $\hat{t} \in t{\uparrow}P$ and $\hat{t}' \in t'{\uparrow}P$ for some $t \sim t'$. A transition over P is *unary* if it lies in the P-closure of a unary transaction.

Namely, the closure over $P \supseteq Q$ of a machine transaction t over Q includes all transitions t' over P in which members of Q do the same in t and in t', and the rest remain in their current (arbitrary) state. The closure likewise carries any relation on transactions to one on the induced transitions. If distinct transactions in R have disjoint P-closures—as when every participant of every transaction in R changes state—then each transition in $R{\uparrow}P$ has a unique inducing transaction, $\sim{\uparrow}P$ relates two transitions exactly when their inducing transactions are $\sim$-related, and $\sim{\uparrow}P$ is a partial equivalence whenever $\sim$ is. For volition-guarded transactions disjointness is required only of unguarded ones (Definition A.9).

A set R of volition-guarded transactions with an equivalence $\sim$ on their underlying machine transactions induces a volitional multiagent transition system:

Definition A.9 (Volitional Multiagent Transition System [30]). Given agents $P \subset \Pi$, machine states S with initial state s_0, a set R of volition-guarded transactions such that every $(t, Q') \in R$ has the participants of t contained in P, and an equivalence $\sim$ on the set $T_R := \{t : (t, Q') \in R \text{ for some } Q'\}$ of underlying machine transactions such that two of them of different classes, each underlying an unguarded transaction of R, have disjoint P-closures, the *volitional multiagent transition system induced by $(S, R, \sim)$ over P* is the multiagent transition system $(\mathcal{A}^P, c_0, T_V, \sim_V)$ where:

(1) $\mathcal{A} := 2^{T_R/\sim} \times S$ is the *agent state space*;
(2) $c_0 \in \mathcal{A}^P$ is the *initial agent configuration*, with ${c_0}^v_p = \emptyset$ and ${c_0}^m_p = s_0$ for every $p \in P$;
(3) T_V consists of all transitions $e \to e' \in (\mathcal{A}^P)^2$ of one of two forms: (*i*) a *volition change* of some $p \in P$—$e^v_p, {e'}^v_p \subseteq (T_R/\sim)_p$ and $e^v_p \neq {e'}^v_p$, $e^m_p = {e'}^m_p$, and $e_r = e'_r$ for every $r \in P \setminus \{p\}$; or (*ii*) a *volitional machine transaction* induced by some volition-guarded transaction $(t, Q') \in R$ per Definition A.4(2);
(4) $\sim_V$ is the restriction of $\sim{\uparrow}P$ (Definition A.8) to the volitional machine transactions, relating two of them whenever their inducing machine transactions are $\sim$-equivalent, and leaves every volition-change transition outside its domain, so volition changes belong to no class and carry no liveness obligation.

Every volitional machine transaction has $\sim$-equivalent inducing machine transactions, so $\sim_V$ is a partial equivalence. A step induced by a guarded transaction (t, Q'), $Q' \neq \emptyset$, removes $[t]$ from the volitional state of each guard, which held it, and a step of another class removes another class, so two guarded transactions of different classes may share their P-closures; two unguarded transactions of different classes have no transition in common, by the condition on P-closures.

Definition A.10 (Enabled [30]). Given a set of volition-guarded transactions, each (t, Q') with $t = d \to d'$ a machine transaction over some $Q \subseteq P$ and S with guards $Q' \subseteq Q$, and an equivalence $\sim$ on machine transactions: the volition-guarded transaction (t, Q') is *machine-enabled* in agent configuration c over P if $c^m_p = d_p$ for every $p \in Q$, and *enabled* if in addition $[t] \in c^v_q$ for every $q \in Q'$. An equivalence class $[t]$ is *enabled* in c if some volition-guarded transaction (t', Q') with $t' \in [t]$ is enabled in c.

A volition-guarded transaction with an empty guard requires no volitions and is enabled whenever its machine precondition is met. A class of volitional machine transactions is enabled at a configuration in the sense of Definition A.5 exactly when some volition-guarded transaction inducing it is enabled in the above sense, and this determines which runs are live and correct. Volition changes belong to no class and so impose no liveness obligation; personal choices remain free.

Protocols and grassroots protocols. A protocol is a family of multiagent transition systems, one for each set of agents $P \subset \Pi$, which share an underlying set of local machine states $\mathcal{S}$ with a designated initial state s_0. A *local-states function* maps every set of agents $P \subset \Pi$ to an arbitrary set of local machine states $S(P) \subset \mathcal{S}$ that includes s_0 and satisfies $P \subset P' \subset \Pi \implies S(P) \subset S(P')$.

Definition A.11 (Protocol [30]). A *protocol $\mathcal{F}$* over a local-states function S is a family of multiagent transition systems that has exactly one transition system $\mathcal{F}(P) = (C(P), c_0(P), T(P), \sim(P))$ for every $P \subset \Pi$, with *agent states* $\mathcal{A}(P)$, configurations $C(P) := \mathcal{A}(P)^P$, initial configuration $c_0(P) \in C(P)$, and partial equivalence $\sim(P)$ on $T(P)$ determined by the protocol, such that $P \subseteq P' \subset \Pi$ implies $\mathcal{A}(P) \subseteq \mathcal{A}(P')$ and $c_0(P)_p = c_0(P')_p$ for every $p \in P$.

Definition A.12 (Interleaving [30]). Let $P, P' \subset \Pi$ be disjoint nonempty sets of agents, $r = c_0, c_1, \ldots$ a run of $\mathcal{F}(P)$, and $r' = d_0, d_1, \ldots$ a run of $\mathcal{F}(P')$. An *interleaving* of r and r' is a sequence

$e_0, e_1, \ldots$ of configurations in $C(P \cup P')$ for which there exist non-decreasing sequences of indices $(i_k)_{k\geq 0}$ and $(j_k)_{k\geq 0}$ with $i_0 = j_0 = 0$ such that for every $k \geq 0$:

(1) $(e_k)_p = (c_{i_k})_p$ for every $p \in P$,
(2) $(e_k)_q = (d_{j_k})_q$ for every $q \in P'$,
(3) if e_{k+1} exists, then exactly one of: (*i*) $i_{k+1} = i_k + 1$ and $j_{k+1} = j_k$ (a P-step), or (*ii*) $i_{k+1} = i_k$ and $j_{k+1} = j_k + 1$ (a P'-step).

Moreover, if r is finite of length n then $i_k = n$ for some k, and if r is infinite then for every $m \geq 0$ there is a k with $i_k = m$; likewise for r' and (j_k).

Definition A.13 (Interaction, Interactive Run, First Interaction [30]). Let $Q \subset \Pi$. For a configuration c over Q and $R \subseteq Q$, write $c|_R$ for the restriction of c to R. A transition $e \to e'$ of $\mathcal{F}(Q)$ is an *interaction between* $x \neq y \in Q$ if $e|_{\{x\}} \neq e'|_{\{x\}}$ and $e|_{\{y\}} \neq e'|_{\{y\}}$, and $e|_{\{x\}} \to e'|_{\{x\}}$ is not a transition of $\mathcal{F}(\{x\})$ or $e|_{\{y\}} \to e'|_{\{y\}}$ is not a transition of $\mathcal{F}(\{y\})$. It is an *interaction between* disjoint nonempty $P, P' \subseteq Q$ if it is an interaction between some $x \in P$ and some $y \in P'$. A run $\hat{r}$ of $\mathcal{F}(Q)$ is *interactive between* P *and* P' if some transition of $\hat{r}$ is an interaction between them; the *first interaction* of P and P' in $\hat{r}$ is the first such transition.

Definition A.14 (Oblivious, Interactive, Grassroots [30]). A protocol $\mathcal{F}$ is:

(1) *oblivious* if for every disjoint nonempty $P, P' \subset \Pi$, every interleaving of a correct run of $\mathcal{F}(P)$ and a correct run of $\mathcal{F}(P')$ is a correct run of $\mathcal{F}(P \cup P')$.
(2) *interactive* if for every $Q \subset \Pi$ and disjoint nonempty $P, P' \subseteq Q$, every prefix of a correct run of $\mathcal{F}(Q)$ containing no interaction between P and P' is a prefix of a correct run of $\mathcal{F}(Q)$ that contains one.
(3) *grassroots* if it is oblivious and interactive.

Being oblivious means two disjoint groups coexist without interference: any interleaving of a correct run of one with a correct run of the other is a correct run of the combined system. Being interactive means two groups that have not yet interacted can still do so: the combined system has a correct continuation in which a single transaction engages agents of both groups. Agents that have taken part in a common interaction are joined; an instance is a maximal set of agents joined directly or indirectly, and two instances coalesce when they come to be one.

Definition A.15 (Interaction Graph, Instance, Coalescence [30]). Let $\mathcal{F}$ be a protocol, $Q \subset \Pi$, and r a prefix of a run of $\mathcal{F}(Q)$. The *interaction graph* of r has vertices Q and an edge between $a \neq b \in Q$ whenever some transition of r is an interaction between a and b. A nonempty $P \subseteq Q$ is an *instance* in r if it is a connected component of the interaction graph of r. Disjoint nonempty $P, P' \subseteq Q$ *coalesce* in a run $\hat{r}$ of $\mathcal{F}(Q)$ if they are instances in some prefix of $\hat{r}$ and lie in one component of the interaction graph of a longer prefix of $\hat{r}$.

Definition A.16 (Transactions Over a Local-State Function [30]). Let S be a local-states function. A set of transactions R is *over* S if every transaction $t \in R$ is a multiagent transition over Q and $S(P')$ for some $Q \subseteq P' \subset \Pi$. Given such a set R and $P \subset \Pi$, $R(P) := \{t \in R : t \text{ is over } Q \text{ and } S(P'), Q \subseteq P' \subseteq P\}$.

Definition A.17 (Volitional Transactions-Based Protocol [30]). Let S be a local-states function and R a set of volition-guarded transactions over S with equivalence $\sim$. The *protocol* $\mathcal{F}$ *over* R, S, *and* $\sim$ assigns to each set of agents $P \subset \Pi$ the volitional multiagent transition system $\mathcal{F}(P)$ induced by $(S(P), R(P), \sim)$ over P (Definition A.9). In particular, $C(P) = \mathcal{A}(P)^P$ with agent state space $\mathcal{A}(P) := 2^{T_{R(P)}/\sim} \times S(P)$—monotone in P, as Definition A.11 requires—and $c_0(P)$ has p-component $(\emptyset, s_0)$ for every $p \in P$.

The obliviousness results rest on the following invariant: in a group's own system an agent wills only transactions internal to the group, so the guard of a transaction that reaches another group cannot will it while the group runs alone, and the transaction is never enabled in an interleaving.

Lemma A.18 (Volitional Containment [30]). *Let $\mathcal{F}$ be a volitional transactions-based protocol over a set of volition-guarded transactions R with equivalence $\sim$. For every $P \subset \Pi$, every configuration c of $\mathcal{F}(P)$, and every $p \in P$: $c_p^v \subseteq T_{R(P)}/\sim$, where $T_{R(P)} := \{t : (t, Q') \in R(P)$ for some $Q'\}$.*

Lemma A.19 (Interleaving Safety [30]). *Let $\mathcal{F}$ be a volitional transactions-based protocol and $P, P' \subset \Pi$ disjoint and nonempty. Every finite prefix of an interleaving of a correct run of $\mathcal{F}(P)$ and a correct run of $\mathcal{F}(P')$ is a finite safe run of $\mathcal{F}(P \cup P')$.*

Proposition A.20 ([30]). *A volitional transactions-based protocol is oblivious provided that for every disjoint nonempty $P, P' \subset \Pi$, no equivalence class whose transactions have participants spanning both P and P' is ever enabled in any interleaving of correct runs of $\mathcal{F}(P)$ and $\mathcal{F}(P')$.*

Corollary A.21 (Guarded Obliviousness [30]). *A volitional transactions-based protocol over a set of volition-guarded transactions R is oblivious if every volition-guarded transaction $(t, Q') \in R$ whose machine transaction t has two or more participants has a nonempty guard, $Q' \neq \emptyset$.*

A person may die or become incapacitated, and an AI agent operating a machine may be retired; the agent has then left, and its will never again has effect. Leaving is made explicit, and interactivity is required only of groups each holding an agent that has not left.

Definition A.22 (Left [30]). Let $\mathcal{F}$ be a volitional transactions-based protocol over $R, Q \subset \Pi$, c a configuration of $\mathcal{F}(Q)$, and $x \in Q$. Then x *has left* in c if no $(t, Q') \in R(Q)$ with $x \in Q'$ is machine-enabled in any configuration c' of $\mathcal{F}(Q)$ with $c \xrightarrow{*} c' \subseteq T(Q)$.

Definition A.23 (Interactive Among the Agents That Have Not Left [30]). A volitional transactions-based protocol $\mathcal{F}$ is *interactive among the agents that have not left* if for every $Q \subset \Pi$ and disjoint nonempty $P, P' \subseteq Q$, every prefix of a correct run of $\mathcal{F}(Q)$ containing no interaction between P and P', in whose last configuration some agent of P and some agent of P' have not left, is a prefix of a correct run of $\mathcal{F}(Q)$ that contains one.

An interactive protocol is interactive among the agents that have not left, the latter requiring a continuation of fewer prefixes. In a run of $\mathcal{F}(P \cup P')$ a member may will a transaction reaching the other group, which a group's own run forbids; the volitional notion constrains the resulting coupling:

Definition A.24 (Volitionally Grassroots [30]). A volitional transactions-based protocol $\mathcal{F}$ is *volitionally grassroots* if it is

oblivious, interactive among the agents that have not left, and, for every disjoint nonempty $P, P' \subset \Pi$ and every safe run of $\mathcal{F}(P \cup P')$ interactive between P and P', the first interaction of P and P' is induced by a volition-guarded transaction (t, Q') with $Q' \cap P \neq \emptyset$ and $Q' \cap P' \neq \emptyset$.

Two groups thus become connected only by an act a person of each wills.

B SPEECH-ACT SCHEMAS

The speech-act schemas [17] of GDO-L and of the restrictions of GFF, written as GFF's are in Section 3.2. GFF-I, GFF-PS, GFF-FR and GFF-FR-BC each have the schemas of the contract they restrict or extend; their boxes show the schemas they replace or add.

B.1 GDO-L

GDO-L: Speech-Act Schemas

GSG:1. *friend*(Alice, Bob), *pointer*(Alice, Bob, Bob), *friend*(Bob, Alice), *pointer*(Bob, Alice, Alice) ←
knows(Alice, Bob), ¬*friend*(Alice, Bob), *knows*(Bob, Alice), ¬*friend*(Bob, Alice) guarded by Alice, Bob.

GSG:2. ¬*friend*(Alice, Bob), ¬*pointer*(Alice, Bob, Bob), ¬*friend*(Bob, Alice), ¬*pointer*(Bob, Alice, Alice) ←
friend(Alice, Bob), *pointer*(Alice, Bob, Bob), *friend*(Bob, Alice), *pointer*(Bob, Alice, Alice) guarded by Alice.

1. *pointer*(Alice, Bob, Dave) ←
friend(Alice, Bob), ¬*pointer*(Alice, Bob, Dave), *pointer*(Bob, Carol, Dave) guarded by Alice.
2. *pointer*(Alice, Carol, Dave) ←
knows(Alice, Carol), *pointer*(Alice, Bob, Dave), ¬*pointer*(Alice, Carol, Dave), ¬(*friend*(Alice, Carol) ∧ Carol=Dave), *pointer*(Bob, Carol, Dave) guarded by Alice.

   pointer(Alice, Carol, Dave) ←
knows(Alice, Carol), *friend*(Alice, Bob), ¬*pointer*(Alice, Carol, Dave), ¬(*friend*(Alice, Carol) ∧ Carol=Dave), *pointer*(Bob, Carol, Dave) guarded by Alice.
3. ¬*pointer*(Alice, Bob, Dave) ←
pointer(Alice, Bob, Dave), ¬(*friend*(Alice, Bob) ∧ Bob=Dave) guarded by Alice.

GSG:1 and GSG:2 are GSG's *befriend* and *unfriend*, recording and removing the pointers friends have. GDO-L:1 is *follow_friend*, adding a Dave-pointer to Bob, who has a Dave-pointer to Carol. GDO-L:2 has two: *follow_transitive*, through a pointer to Bob, and *follow_transitive_friend*, through Alice's friend Bob; GDO-L:3 is *unfollow*. In each of the four Bob neither wills the act nor changes state, and the act reads Bob's state.

B.2 GFF-I

GFF-I: Speech-Act Schemas

1. *pointer*(Alice, Carol, Dave) ←
knows(Alice, Carol), *pointer*(Alice, Bob, Dave), ¬*pointer*(Alice, Carol, Dave), ¬(*friend*(Alice, Carol) ∧ Carol=Dave), *pointer*(Bob, Carol, Dave), *knows*(Carol, Alice) guarded by Alice.

   pointer(Alice, Carol, Dave) ←
knows(Alice, Carol), *friend*(Alice, Bob), ¬*pointer*(Alice, Carol, Dave), ¬(*friend*(Alice, Carol) ∧ Carol=Dave), *pointer*(Bob, Carol, Dave), *knows*(Carol, Alice) guarded by Alice.
2. *item*(Alice, x, Carol), *received*(Alice, x, Carol, Bob) ←
knows(Bob, Alice), *item*(Bob, x, Carol), *pointer*(Alice, Bob, Carol), ¬*item*(Alice, x, Carol).

The two schemas of GFF-I:1 add the reading role Carol, who knows Alice. Bob, the first role of delivery, requires that Bob knows Alice and holds the item.

B.3 GFF-PS

GFF-PS: Speech-Act Schemas

1. *pointer*(Alice, Carol, Dave), ¬*pointer*(Alice, Bob, Dave) ←
knows(Alice, Carol), *pointer*(Alice, Bob, Dave), ¬*pointer*(Alice, Carol, Dave), ¬(*friend*(Alice, Carol) ∧ Carol=Dave), *pointer*(Bob, Carol, Dave), *knows*(Carol, Alice).

The act has no guarding role, and Bob and Carol are reading roles; its signature atoms [17] are the pointer to Carol it adds and the pointer to Bob it deletes, which together name Bob and Carol.

B.4 GFF-FR

GFF-FR: Speech-Act Schemas

1. *pointer*(Alice, Bob, Carol) ←
friend(Alice, Bob), ¬*pointer*(Alice, Bob, Carol), *pointer*(Alice, c, Carol), *redundancy*(Alice, d), *pointer*(Bob, e, Carol), cardinality(f, *pointer*(Alice, f, Carol)) < d.

The act has the head and the roles of *follow_friend* and a stronger body, and no guarding role; Bob is a reading role, and the pointer to Bob it adds is its signature atom [17]. d is the value of an environment predicate, *redundancy*, which the platform keeps; c and e are agent variables; and the cardinality counts Alice's Carol-pointers [17].

B.5 GFF-FR-BC

GFF-FR-BC: Speech-Act Schemas

1. *item*(Carol, x, Dave), *received_back*(Carol, x, Dave, Alice) ←
   pointer(Alice, Bob, Dave), *pointer*(Alice, Carol, Dave), *item*(Alice, x, Dave), *received*(Alice, x, Dave, Bob), ¬*item*(Carol, x, Dave).

The act has no guarding role; Alice, its first role, is a reading role, and Carol records the item and its sender with *received_back*, its signature atom, *received* being delivery's [17].

C CERTIFICATION

Proposition C.1 (Certification). *GFF is syntactically grassroots.*

Proof. By the three conditions of [17]. *Unobstructed introductory act. acquaint* is unobstructed as in GKK [17]: it requires *identity*, which *establish* adds, and forbids nothing. *Traceable provenance* of the set of all predicates. GKK's schemas meet it [17]; *befriend* adds at each role *friend* and *pointer* naming the other role, and *unfriend* adds nothing. *point_friend* adds at Alice an atom whose argument is the role Bob; *point_transitive* adds *points_to*(Carol) at Alice, who requires *knows*(Carol); *follow_friend* adds *pointer*(Bob, Dave) at Alice, Dave an argument of *pointer*(Carol, Dave), which Bob requires; *follow_transitive* adds *pointer*(Carol, Dave) at Alice, who requires *knows*(Carol) and *pointer*(Bob, Dave), and so does *follow_transitive_friend*, Alice requiring *knows*(Carol) and Bob *pointer*(Carol, Dave); *post* adds *item*(x, Alice), naming its own role, x occurring in no required atom; and *deliver* adds *item*(x, Carol) and *received*(x, Carol, Bob) at Alice, who requires *pointer*(Bob, Carol), x an argument of *item*(x, Carol), which Bob requires; *unpoint* and *unfollow* add nothing. *Volition. post* has arity one. In each schema of GDO and GDO-L Alice guards and requires an atom naming Bob, the reading role: *friend*(Bob), *points_to*(Bob) or *pointer*(Bob, Dave), each of traceable provenance. *deliver* has no guarding role and the reading role Bob: its role graph is connected, Alice requiring *pointer*(Bob, Carol), and *received*(x, Carol, Bob) is its signature atom, naming Bob, Carol and x, the only atom of its predicate that a schema adds. □

[TBD: The checker's verdict on the six contracts of GLP main c9fc8dbf.]

D THE TRANSACTIONS AND THE PROOFS

Fourteen schemas, one set of transactions each, determined by the schema at a binding (Definition Denotation of [17]): the participants are the persons the binding sends the roles to, the guards those in the guarding roles, and at each participant the transaction requires, forbids, adds and deletes what the role does, *identity* required at every participant. GKK's and GSG's are those of [17]. Those of GDO-L and GFF are each written from its schema in Section 3.2 or Appendix B.

Transaction equivalence. The equivalence ~ is that of [17], generated by relating two transactions of the same participants that one schema at one binding determines. So the deliveries of one item of one feed to a from b form one class.

The records. *item*(x, l) is added at a person only by *post*, with l the poster, and by *deliver*, each of which forbids it there, and no schema deletes it. *received*(x, l, b) is added only by *deliver*, with the item. *pointer*(q, l) is added at a person only by an act that person guards, *follow_friend*, *follow_transitive* or *follow_transitive_friend*, and deleted only by *unfollow*, which that person guards. *pointer*(q, q) is added at a person also by *befriend*, with *friend*(q), and deleted by *unfriend*, which that person guards or their friend does.

Proposition 3.1 (Provenance of a Feed). *In every run of the feeds-and-followers protocol, a person a holds an item x of the feed of l only if l posted x and signed it; if $a \neq l$, then a received x from a person who held it as an item of the feed of l and to whom a had an l-pointer; and a holds x once.*

Proof. By induction on the run. The initial state holds no *item* atom. A transition adds *item*(x, l) at a by *post* or by *deliver*. If by *post*(a; x), then $l = a$, so l posted x, and x, occurring in no required atom, is signed by a, the person in the guarding role [17]. If by *deliver*(a, b; l, x), then b holds *item*(x, l) and a holds *pointer*(b, l), so a has an l-pointer to b, receives x from b, who held it as an item of that feed, and by the induction hypothesis l posted x and signed it; here $a \neq l$, since l holds every item they posted and the delivery forbids the item at a. Both acts forbid *item*(x, l) at a, so a holds it once. □

Proposition 3.2 (Delivery). *In every correct run of the feeds-and-followers protocol, if from some configuration on a has an l-pointer to b and b holds an item x of the feed of l, then a comes to hold x. Hence if l posted x and $l = p_0, p_1, \ldots, p_n = a$ are persons such that, from the posting on, p_i has an l-pointer to p_{i-1} for every $1 \leq i \leq n$, then a comes to hold x.*

Proof. Let a have an l-pointer to b and b hold *item*(x, l), from a configuration c on. No schema deletes an *item* atom. While a does not hold *item*(x, l), the transaction *deliver*(a, b; l, x) is machine-enabled, and, being unguarded, enabled; so its class is enabled in every configuration from c on in which a does not hold the item. A correct run is live (Definition A.5), so either a member of the class is taken, after which a holds x, or a comes to hold x by another act. For the second part, by induction on i, p_i comes to hold x as an item of the feed of l: $p_0 = l$ holds it from the posting on, and once p_{i-1} holds it, the first part applies to p_i and p_{i-1}. □

Proposition 3.4 (Certification of GFF-I). *GFF-I is syntactically grassroots and implementable.*

Proof. GFF-I is obtained from GFF, which is syntactically grassroots (Proposition C.1), by replacing three schemas by schemas with their names, heads and guards and stronger bodies, adding a reading role, Carol, to each of the two schemas of GDO-L:2, which have a guarding role. So GFF-I is syntactically grassroots if and only if it has an unobstructed introductory act and Carol has an edge to a guarding role from an atom that guarding role requires [17, Theorem What the check preserves]. *acquaint* is kept, requires *identity*, which the kept *establish* adds, and forbids nothing, so it

is unobstructed in GFF-I; and Alice, the guarding role of each, requires *knows*(Alice, Carol). *deliver* is the one schema of GFF-I with no guarding role; its first role, Bob, requires *knows*(Bob, Alice) and *item*(Bob, x, Carol), which name Alice, the one other role, and x and Carol, the variables of the atoms it adds. So GFF-I is implementable. □

Lemma D.1 (Bob knows Alice). *In every run of the protocol realising GFF-I, if Alice has a Dave-pointer to Bob, then Bob holds knows*(Alice).

Proof. No schema deletes a *knows* atom, so it suffices that Bob holds *knows*(Alice) when Alice comes to have a Dave-pointer to Bob. *pointer*(Bob, Dave) is added at Alice by *befriend*, which requires *knows*(Alice) at Bob, by *follow_friend*, which requires *friend*(Bob) at Alice, and by *follow_transitive* and *follow_transitive_friend*, which in GFF-I require *knows*(Alice) at the person pointed to. A *friend*(Bob) atom is added at Alice only by *befriend*, which requires *knows*(Alice) at Bob. □

Proposition 3.5 (Restriction). *GFF-I restricts GFF.*

Proof. By Proposition Syntactic restrictions of [17], in two steps. GFF and GFF-I are syntactically grassroots (Propositions C.1 and 3.4), and so are the contracts between them below, by the proof of Proposition 3.4; the three have simple unguarded classes, their one schema with no guarding role, *deliver*, adding *received*(Alice, x, Carol, Bob), the only atom of its predicate that a schema adds, deletes or clears, which names every role and every variable of the schema [17]. First, by (a) twice, *follow_transitive* and *follow_transitive_friend* are replaced by GFF-I's, each with its name and guard and a stronger body, Carol being in each an added agent role that is a reading role. Then, by (c), *deliver* is replaced by GFF-I's, with its name, roles and guards and a stronger body, the added atom *knows*(Bob, Alice): at every configuration of every run of the protocol realising GFF-I and every binding at which GFF's body holds, Alice has a Carol-pointer to Bob, so by Lemma D.1 Bob holds *knows*(Alice), and GFF-I's body holds. □

Proposition 4.1 (Certification of GFF-PS). *GFF-PS is syntactically grassroots and implementable.*

Proof. GFF-PS amends GFF-I, which is syntactically grassroots (Proposition 3.4), adding *shorten*, so by Theorem What the check preserves of [17], clause 1, it suffices that GFF-PS has an unobstructed introductory act, that the atoms *shorten* adds have traceable provenance, that *shorten* meets Volition, and that *deliver* keeps its signature atoms. *acquaint* is kept and forbids nothing, and *establish*, which adds the *identity* it requires, is kept, so it is unobstructed. *shorten* adds *pointer*(Carol, Dave) at Alice: Carol is a role, holding *knows*(Carol, Alice), and Dave is an argument of *pointer*(Bob, Dave), which Alice requires. *shorten* has no guarding role; its role graph is connected, Alice requiring *knows*(Carol) and *pointer*(Bob, Dave); its reading roles, Bob and Carol, are agent roles; and *pointer*(Carol, Dave), the one atom it adds, and *pointer*(Bob, Dave), the one atom it deletes, both at Alice, are its signature atoms, no other schema with no guarding role adding or deleting an atom of that predicate, and together they name Bob and Carol. *received* stays the signature atom of *deliver*. So GFF-PS is syntactically grassroots. It is implementable: the atoms *shorten*'s first role, Alice, requires name Bob, Carol and Dave, every other role and every variable of the atoms it adds and deletes, and *deliver* is as in GFF-I. □

Proposition 4.2 (Path shortening). *GFF-PS restricts GFF-I.*

Proof. Write $\mathcal{F}$ and $\mathcal{F}'$ for the protocols realising GFF-I and GFF-PS, and let r' be a correct run of $\mathcal{F}'(P)$. Let r start at the initial configuration of $\mathcal{F}(P)$ and follow r' as follows. A volition change of r' is left out. A volitional machine transaction of r' induced by (t, Q') for a schema of GFF-I is taken in r after a volition change of each guard in Q' adding the class of t, which the transaction removes from every person (Definition A.4). A transaction of *shorten* at Alice, Bob, Carol and Dave is taken in r in two: a volition change of Alice and the transaction of *follow_transitive* at the same persons, whose body is *shorten*'s, adding *pointer*(Carol, Dave) at Alice; then a volition change of Alice and the transaction of *unfollow* at Alice, Bob and Dave, whose body *pointer*(Bob, Dave) at Alice still holds, deleting it. The two reach the machine configuration *shorten* reaches. So r is a safe run of $\mathcal{F}(P)$ whose machine configurations include those of r', in order, from the first and, where these are finitely many, to the last, and in which a person wills nothing but the class of the next transaction, until it is taken.

r is live. A class of guarded transactions is enabled in r only once its guards will it, and is taken at the next step. The one schema of GFF-I with no guarding role is *deliver*, and each of its classes is the set of transactions it determines at one binding, in GFF-I and in GFF-PS alike, *received* being added by no other schema. If such a class is enabled in every configuration of a suffix of r, it is enabled in those of r' in that suffix, which form a suffix of r', and r', being live, takes a member of it, which r takes too. So r is correct, and GFF-PS restricts GFF-I [17, Definition Implementation, restriction]. □

Proposition 5.1 (Certification of GFF-FR). *GFF-FR is syntactically grassroots and implementable.*

Proof. GFF-FR amends GFF-I adding *redundant_follow*, so by Theorem What the check preserves of [17], clause 1, as for GFF-PS. *acquaint* is kept and unobstructed, requiring no environment atom. *redundant_follow* adds *pointer*(Bob, Carol) at Alice: Bob is a role, and Carol is an argument of *pointer*(e, Carol), which Bob requires. It has no guarding role; its role graph is connected, Alice requiring *friend*(Bob); its reading role, Bob, is an agent role; and *pointer*(Bob, Carol), the one atom it adds, is its signature atom, no other schema with no guarding role adding an atom of that predicate, and names Bob. *received* stays the signature atom of *deliver*. So GFF-FR is syntactically grassroots. It is implementable: the atoms its first role, Alice, requires name Bob and Carol, the one other role and the one variable of the atom it adds. □

Proposition 5.2 (Fault resilience). *GFF-FR restricts GFF-I.*

Proof. GFF-FR is obtained from GFF-I by adding *redundant_follow*, a schema with no guarding role, with the head and the roles of *follow_friend*, Dave written Carol, and a stronger body: it requires at each role every atom *follow_friend* requires there, *friend*(Bob) at Alice and a Carol-pointer at Bob, forbids what it forbids, and requires besides a Carol-pointer of Alice's, the environment atom

redundancy(d) and the constraint. GFF-I and GFF-FR are syntactically grassroots (Propositions 3.4 and 5.1), so GFF-FR restricts GFF-I by Proposition Syntactic restrictions of [17], (b). □

Proposition 5.3 (Certification of GFF-FR-BC). *GFF-FR-BC is syntactically grassroots and implementable.*

Proof. GFF-FR-BC amends GFF-FR, which is syntactically grassroots (Proposition 5.1), adding *back_forward*, so by Theorem What the check preserves of [17], clause 1, as for GFF-PS. *acquaint* is kept and unobstructed. *back_forward* adds *item*(x, Dave) and *received_back*(x, Dave, Alice) at Carol: Alice is a role, and x and Dave are arguments of *item*(x, Dave), which Alice requires. It has no guarding role; its role graph is connected, Alice requiring *pointer*(Carol, Dave); its reading role, Alice, is an agent role; and *received_back*(x, Dave, Alice), the one atom of its predicate it adds, no other schema adding one, is its signature atom and names Alice. The kept schemas with no guarding role, *deliver* and *redundant_follow*, keep their signature atoms. So GFF-FR-BC is syntactically grassroots. It is implementable: the atoms its first role, Alice, requires name Carol, Bob, Dave and x, every other role and every variable of the atoms it adds. □

Proposition 5.4 (Back-channel). *In every correct run of the protocol realising GFF-FR-BC, if from some configuration on a has l-pointers to b and to c and holds an item x of the feed of l received from b, then c comes to hold x. In every run of it, a person a holds an item x of the feed of l only if l posted x and signed it; if $a \neq l$, then a received x from a person to whom a had an l-pointer, or who had an l-pointer to a; and a holds x once.*

Proof. For the first part, let a have l-pointers to b and to c and hold *item*(x, l) and *received*(x, l, b) from a configuration on; no schema deletes either. While c does not hold *item*(x, l), the transaction of *back_forward* at a, c, b, l and x is machine-enabled and, being unguarded, enabled, so in a correct run c comes to hold x, by it or by another act. For the second, by induction on the run, as in the proof of Proposition 3.1: *item*(x, l) is added at a by *post*, with $l = a$; by *deliver*, from a person to whom a has an l-pointer; or by *back_forward*, from a person who has an l-pointer to a and held the item; each forbids *item*(x, l) at a, so a holds it once. □

Proposition 3.3 (Grassroots Property). *The feeds-and-followers protocol is volitionally grassroots.*

Proof. The contract is syntactically grassroots (Proposition C.1), so the protocol realising it is volitionally grassroots by Theorem Grassroots realisation of [17]. □